

# 9. АСИНХРОННОЕ ПРОГРАММИРОВАНИЕ (ASYNCHRONOUS PROGRAMMING)

---

Асинхронное программирование (asynchronous programming), как следует из названия, — это программирование, которое не является синхронным. На высоком уровне асинхронная операция (asynchronous operation) — это такая операция, которая выполняется в фоновом режиме: программа не ждёт завершения асинхронной операции, а вместо этого немедленно переходит к следующей строке кода. Если вы ещё не знакомы с асинхронным программированием, такое определение может показаться недостаточным, поскольку оно на самом деле не объясняет, чем именно *является* асинхронное программирование. Чтобы по-настоящему понять модель асинхронного программирования и то, как она работает в Rust, нам сначала нужно разобраться, какова альтернатива. То есть нам нужно понять модель *синхронного* (synchronous) программирования, прежде чем мы сможем понять *асинхронную* (asynchronous). Это важно как для прояснения концепций, так и для демонстрации компромиссов, связанных с использованием асинхронного программирования: асинхронное решение не всегда является правильным! Мы начнём эту главу с краткого путешествия по тому, что в первую очередь мотивирует асинхронное программирование как концепцию; затем мы углубимся в то, как асинхронность в Rust на самом деле работает под капотом.

## В чём суть асинхронности? (What's the Deal with Asynchrony?)

Прежде чем перейти к деталям синхронной и асинхронной моделей программирования, нам сначала нужно бегло взглянуть на то, что на самом деле делает ваш компьютер, когда выполняет ваши программы.

Компьютеры быстры. Действительно быстры. Настолько быстры, что большую часть времени они проводят в ожидании, пока что-то произойдёт. Если только вы не распаковываете файлы, не кодируете аудио или не перемалываете числа, скорее всего, ваш процессор в основном простаивает, ожидая

завершения операций. Он ждёт прибытия сетевого пакета, движения мыши, окончания записи нескольких байтов на диск или, может быть, даже просто чтения из основной памяти. С точки зрения процессора, между большинством таких событий проходят целые эпохи. Когда же одно из них происходит, процессор выполняет несколько инструкций, а затем снова возвращается к ожиданию. Взгляните на загрузку вашего процессора — вероятно, она где-то в районе нескольких процентов, и, скорее всего, именно там она держится большую часть времени.

## **Синхронные интерфейсы (Synchronous Interfaces)**

Синхронные интерфейсы (synchronous interfaces) позволяют вашей программе (точнее, одному потоку в вашей программе) выполнять только одну операцию за раз; каждая операция должна дожидаться завершения предыдущей синхронной операции, прежде чем сможет запуститься. Большинство интерфейсов, которые вы встречаете на практике, синхронны: вы вызываете их, они что-то делают и в конце концов возвращают управление, когда операция завершена и ваша программа может продолжаться дальше. Причина этого, как мы увидим позже в этой главе, в том, что превращение операции в асинхронную требует немалого количества дополнительной машинерии. Если только вам не нужны преимущества асинхронности, придерживаться синхронной модели гораздо проще и требует куда меньше церемоний.

Синхронные интерфейсы скрывают всё это ожидание; приложение вызывает функцию, которая говорит «запиши эти байты в этот файл», и через некоторое время эта функция завершается, и выполняется следующая строка кода. За кулисами же на самом деле происходит то, что операционная система ставит операцию записи в очередь к диску, а затем усыпляет приложение, пока диск не сообщит, что он закончил запись. Приложение воспринимает это как функцию, выполнение которой занимает долгое время, но в действительности она вовсе не выполняется — она просто ждёт.

Интерфейс, который выполняет операции последовательно подобным образом, также часто называют *блокирующим* (blocking), поскольку операция в интерфейсе, которой приходится ждать наступления некоторого внешнего события, чтобы продвинуться вперёд, *блокирует* (blocks) дальнейшее выполнение до тех пор, пока это событие не произойдёт. Называете ли вы интерфейс синхронным или блокирующим, основная идея одна и та же:

приложение не движется дальше, пока текущая операция не завершится. Пока операция ждёт, ждёт и приложение.

Синхронные интерфейсы обычно считаются простыми в работе и понятными для рассуждений, поскольку ваш код выполняется по одной строке за раз. Но они же позволяют приложению делать только что-то одно за раз. Это означает, что если вы хотите, чтобы ваша программа ждала либо пользовательского ввода, либо сетевого пакета, вам не повезло, если только ваша операционная система не предоставляет специальную операцию для этого. Аналогично, даже если ваше приложение могло бы выполнять какую-то другую полезную работу, пока диск записывает файл, у него нет такой возможности, поскольку операция записи в файл блокирует выполнение!

## **Многопоточность (Multithreading)**

На сегодняшний день самое распространённое решение, позволяющее организовать параллельное выполнение, — это использовать *многопоточность* (multithreading). В многопоточной программе каждый поток отвечает за выполнение определённой независимой последовательности блокирующих операций, а операционная система мультиплексирует потоки так, что если хоть какой-то поток может продвинуться вперёд, прогресс достигается. Если один поток блокируется, какой-то другой поток всё ещё может быть готов к выполнению, и поэтому приложение может продолжать делать полезную работу.

Обычно эти потоки общаются друг с другом с помощью примитива синхронизации, такого как блокировка или канал, чтобы приложение всё же могло координировать их усилия. Например, у вас может быть один поток, который ждёт пользовательского ввода, один поток, который ждёт сетевых пакетов, и ещё один поток, который ждёт, пока любой из этих двух потоков отправит сообщение в канал, разделяемый между всеми тремя потоками.

Многопоточность даёт вам *конкурентность* (concurrency) — возможность иметь несколько независимых операций, которые могут выполняться в любой момент времени. Именно система, на которой выполняется приложение (в данном случае операционная система), выбирает среди операций, которые не заблокированы, и решает, какую из них выполнять следующей. Если один поток заблокирован, она может выбрать выполнение другого, который способен продвинуться вперёд.

Многопоточность в сочетании с блокирующими интерфейсами позволяет зайти достаточно далеко, и значительные объёмы готового к продакшену программного обеспечения построены именно таким образом. Но у этого подхода есть свои недостатки. Во-первых, отслеживание всех этих потоков быстро становится обременительным; если вам приходится запускать поток для каждой конкурентной задачи, включая такие простые, как ожидание ввода с клавиатуры, потоки быстро накапливаются, а вместе с ними и дополнительная сложность, необходимая для отслеживания того, как все эти потоки взаимодействуют, общаются и координируются.

Во-вторых, переключение между потоками обходится тем дороже, чем больше потоков. Каждый раз, когда один поток прекращает выполнение, а другой запускается на его месте, вам нужно совершить обход к планировщику операционной системы, а это не бесплатно. На некоторых платформах порождение новых потоков — довольно тяжеловесный процесс. Приложения с высокими требованиями к производительности часто снижают эти затраты, переиспользуя потоки и используя системные вызовы операционной системы, которые позволяют блокироваться на многих связанных операциях, но в конечном счёте вы остаётесь с той же проблемой: блокирующие интерфейсы требуют, чтобы у вас было столько потоков, сколько блокирующих вызовов вы хотите сделать.

Наконец, потоки вносят в вашу программу *параллелизм* (parallelism). Различие между конкурентностью и параллелизмом тонкое, но важное: конкурентность означает, что выполнение ваших задач чередуется, тогда как параллелизм означает, что несколько задач выполняются в одно и то же время. Если у вас есть две задачи, их выполнение, выраженное в ASCII, могло бы выглядеть как `_ _ _ _` (конкурентность) против `=====` (параллелизм). Многопоточность не обязательно подразумевает параллелизм — даже если у вас много потоков, у вас может быть лишь одно ядро, так что в данный момент выполняется только один поток, — но эти две вещи обычно идут рука об руку. Вы можете сделать два потока взаимоисключающими в их выполнении с помощью `Mutex` или другого примитива синхронизации, но это вносит дополнительную сложность. И хотя параллелизм часто хорош — кто не хочет, чтобы его программа работала быстрее на большем числе ядер, — он также означает, что ваша программа должна справляться с действительно одновременным доступом к разделяемым структурам данных. Это означает переход от `Rc`, `Cell` и `RefCell` к более мощным, но и более медленным `Arc` и `Mutex`.

И хотя вы *можете* захотеть использовать последние типы в своей конкурентной программе, чтобы включить параллелизм, использование потоков *вынуждает* вас их применять. Мы рассмотрим многопоточность гораздо подробнее в главе 11.

## Асинхронные интерфейсы (Asynchronous Interfaces)

Теперь, когда мы изучили синхронные интерфейсы, мы можем взглянуть на альтернативу: асинхронные, или *неблокирующие* (nonblocking), интерфейсы. Асинхронный интерфейс — это такой интерфейс, который может не выдать результат сразу же, а вместо этого может сообщить, что результат будет доступен в какой-то более поздний момент времени. Это даёт вызывающему возможность тем временем заняться чем-то другим, вместо того чтобы засыпать до завершения этой конкретной операции. В терминологии Rust асинхронный интерфейс — это метод, который возвращает `Poll`, как определено в листинге 9-1.

```
enum Poll<T> {  
    Ready(T),  
    Pending  
}
```

*Листинг 9-1: Ядро асинхронности — тип «вот вам результат, или возвращайтесь позже»*

`Poll` обычно появляется в типе возвращаемого значения функций, имена которых начинаются с `poll` — это методы, сигнализирующие о том, что они могут попытаться выполнить операцию без блокировки. Мы разберём, как именно они это делают, позже в этой главе, но в общем случае они пытаются выполнить столько работы, сколько возможно, прежде чем они обычно заблокировались бы, а затем возвращают управление. И, что критически важно, они запоминают, на чём остановились, чтобы позже можно было возобновить выполнение, когда снова появится возможность продвинуться.

Эти неблокирующие функции позволяют нам легко выполнять несколько задач конкурентно. Например, если вы хотите читать либо из сети, либо с клавиатуры пользователя — смотря что станет доступно первым, — всё, что вам нужно сделать, это опрашивать (`poll`) оба источника в цикле, пока один из них не вернёт `Poll::Ready`. Никаких дополнительных потоков или

синхронизации не требуется!

Слово *цикл* (loop) здесь должно вас слегка насторожить. Вы же не хотите, чтобы ваша программа прокручивала цикл три миллиарда раз в секунду, когда до следующего события ввода могут быть минуты. В мире блокирующих интерфейсов это не было проблемой, поскольку операционная система просто усыпляла поток, а затем брала на себя его пробуждение при наступлении релевантного события, но как нам избежать прожигания процессорных циклов в ожидании в этом дивном новом неблокирующем мире? Именно об этом и будет большая часть оставшейся части этой главы.

## Стандартизированный опрос (Standardized Polling)

Чтобы прийти к миру, где каждую библиотеку можно использовать в неблокирующей манере, можно было бы заставить каждого автора библиотеки придумывать собственные методы `poll`, все со слегка различающимися именами, сигнатурами и типами возвращаемых значений, — но это быстро стало бы неуправляемым. Вместо этого в Rust опрос (polling) стандартизирован с помощью трейта `Future`. Упрощённая версия `Future` показана в листинге 9-2 (к настоящей версии мы вернёмся позже в этой главе).

```
trait Future {  
    type Output;  
    fn poll(&mut self) -> Poll<Self::Output>;  
}
```

*Листинг 9-2: Упрощённый взгляд на трейт `Future`*

Типы, реализующие трейт `Future`, известны как *футуры* (futures) и представляют значения, которые могут быть пока недоступны. Футура может представлять следующий момент прихода сетевого пакета, следующее движение курсора мыши или просто момент, когда истечёт некоторый промежуток времени. Запись `Future<Output = Foo>` можно читать как «тип, который произведёт `Foo` в будущем». Подобные типы в других языках часто называют *промисами* (promises) — они обещают, что в конце концов выдадут указанный тип. Когда футура в итоге возвращает `Poll::Ready(T)`, мы говорим, что футура *разрешается* (resolves) в `T`.

С этим трейтом на руках мы можем обобщить шаблон предоставления методов

`poll`. Вместо того чтобы иметь методы вроде `poll_recv` и `poll_keypress`, у нас могут быть методы вроде `recv` и `keypress`, которые оба возвращают `impl Future` с подходящим типом `Output`. Это не отменяет того факта, что вам придётся их опрашивать — с этим мы разберёмся позже, — но это означает, что по крайней мере существует стандартизированный интерфейс к подобного рода ожидаемым значениям, и нам не нужно использовать префикс `poll_` повсюду.

**ПРИМЕЧАНИЕ** В общем случае вам не следует опрашивать футуру снова после того, как она вернула `Poll::Ready`. Если вы это сделаете, футура имеет полное право запаниковать. Футуру, которую безопасно опрашивать после того, как она вернула `Ready`, иногда называют слитой футурой (`fused future`).

## Эргономичные футуры (Ergonomic Futures)

Написание типа, реализующего `Future` тем способом, который я описал до сих пор, — довольно мучительное занятие. Чтобы понять почему, сначала взгляните на довольно простой асинхронный блок кода в листинге 9-3, который просто пытается пересылать сообщения из входного канала `rx` в выходной канал `tx`.

```
async fn forward<T>(rx: Receiver<T>, tx: Sender<T>) {
    while let Some(t) = rx.next().await {
        tx.send(t).await;
    }
}
```

*Листинг 9-3: Реализация пересылающей по каналу футуры с помощью `async` и `await`*

Этот код, написанный с использованием синтаксиса `async` и `await`, выглядит очень похоже на его эквивалентный синхронный код и легко читается. Мы просто отправляем каждое полученное сообщение в цикле, пока сообщения не закончатся, и каждая точка `await` соответствует месту, где синхронный вариант мог бы заблокироваться. А теперь подумайте, что было бы, если бы вместо этого вам пришлось выражать этот код, вручную реализуя трейт `Future`. Поскольку каждый вызов `poll` начинается с вершины функции, вам нужно было бы упаковать необходимое состояние, чтобы продолжить с того места, где код остановился в последний раз. Результат получается довольно гротескным, как демонстрирует листинг 9-4.

```

enum Forward<T> { ❶
    WaitingForReceive(ReceiveFuture<T>, Option<Sender<T>>),
    WaitingForSend(SendFuture<T>, Option<Receiver<T>>),
}

impl<T> Future for Forward<T> {
    type Output = (); ❷
    fn poll(&mut self) -> Poll<Self::Output> {
        match self { ❸
            Forward::WaitingForReceive(recv, tx) => {
                if let Poll::Ready((rx, v)) = recv.poll() {
                    if let Some(v) = v {
                        let tx = tx.take().unwrap(); ❹
                        *self = Forward::WaitingForSend(tx.send(v), Some(rx)); ❺
                        // Пытаемся продвинуться с отправкой.
                        return self.poll(); ❻
                    } else {
                        // Больше элементов нет.
                        Poll::Ready(())
                    }
                } else {
                    Poll::Pending
                }
            }
            Forward::WaitingForSend(send, rx) => {
                if let Poll::Ready(tx) = send.poll() {
                    let rx = rx.take().unwrap();
                    *self = Forward::WaitingForReceive(rx.receive(), Some(tx));
                    // Пытаемся продвинуться с приёмом.
                    return self.poll();
                } else {
                    Poll::Pending
                }
            }
        }
    }
}

```

#### Листинг 9-4: Ручная реализация пересылающей по каналу футуры

Вам редко придётся писать код подобный этому в Rust в наши дни, но он даёт важное понимание того, как всё работает под капотом, так что давайте разберём его. Сначала мы определяем наш тип футуры как перечисление `enum` ❶, которое мы будем использовать для отслеживания того, чего мы в данный момент ожидаем. Это следствие того факта, что когда мы возвращаем `Poll::Pending`, следующий вызов `poll` снова начнётся с вершины функции. Нам нужен какой-то способ узнать, на чём мы остановились, чтобы мы знали, какую операцию продолжать. Более того, нам нужно отслеживать разную информацию в зависимости от того, что мы делаем: если мы ждём завершения приёма, нам нужно хранить эту `ReceiveFuture` (её определение не показано в



этом примере), чтобы мы могли опросить её в следующий раз, когда нас самих опросят, и то же самое касается `SendFuture`. Элементы `Option` здесь тоже могут показаться вам странными; к ним мы вскоре вернёмся.

Когда мы реализуем `Future` для `Forward`, мы объявляем его выходной тип как `()` ❷, потому что эта футура на самом деле ничего не возвращает. Вместо этого футура разрешается (без результата) тогда, когда она закончила пересылку всего из входного канала в выходной. В более полном примере `Output` нашего пересылающего типа мог бы быть `Result`, чтобы он мог сообщать об ошибках от `receive()` и `send()` вверх по стеку функции, которая опрашивает завершение пересылки. Но этот код и так достаточно сложен, так что оставим это на другой день.

Когда `Forward` опрашивается, ему нужно возобновиться там, где он в последний раз остановился, что мы выясняем сопоставлением с образцом по варианту перечисления, который в данный момент хранится в `self` ❸. В какую бы ветку мы ни попали, первым шагом будет опрос той футуры, которая блокирует прогресс для текущей операции; если мы пытаемся принять, мы опрашиваем `ReceiveFuture`, а если мы пытаемся отправить, мы опрашиваем `SendFuture`. Если этот вызов `poll` возвращает `Poll::Pending`, то мы не можем продвинуться, и мы сами возвращаем `Poll::Pending`. Но если текущая футура разрешается, у нас есть работа!

Когда одна из внутренних футур разрешается, нам нужно обновить то, какой вариант текущей операции хранится в `self`, переключившись на другой вариант перечисления. Чтобы сделать это, нам нужно переместить значение из `self` для вызова `Receiver::receive` или `Sender::send` — но мы не можем этого сделать, поскольку всё, что у нас есть, — это `&mut self`. Поэтому мы перемещаем нужное нам состояние через `Option::take` ❹. Это выглядит глупо, поскольку мы всё равно собираемся перезаписать `self` ❺, и, следовательно, `Option` всегда будет `Some`, но иногда нужны такие уловки, чтобы удовлетворить проверщик заимствований (`borrow checker`).

Наконец, если мы всё же продвигаемся вперёд, мы затем снова опрашиваем `self` ❻, чтобы, если мы можем сразу продвинуться с ожидающей отправкой или приёмом, мы это сделали. Это на самом деле необходимо для корректности при реализации настоящего трейта `Future`, к которому мы вернёмся позже, но пока что считайте это оптимизацией.

Мы только что вручную написали *машину состояний* (state machine): тип, у которого есть некоторое число возможных состояний и который переходит между ними в ответ на определённые события. И это была ещё довольно простая машина состояний. Представьте, что вам придётся писать подобный код для более сложных вариантов использования, где есть дополнительные промежуточные шаги!

Помимо написания громоздкой машины состояний, нам нужно знать типы футур, которые возвращают `Sender::send` и `Receiver::receive`, чтобы мы могли хранить их в нашем типе. Если бы вместо этого эти методы возвращали `impl Future`, у нас не было бы способа выписать типы для наших вариантов. Методам `send` и `receive` также приходится забирать владение отправителем и получателем; если бы это было не так, времена жизни футур, которые они возвращают, были бы привязаны к заимствованию `self`, которое закончилось бы при возврате из `poll`. Но это не сработало бы, поскольку мы пытаемся хранить эти футуры в `self`.

**ПРИМЕЧАНИЕ** Возможно, вы заметили, что `Receiver` очень похож на асинхронную версию `Iterator`. Другие заметили то же самое, и стандартная библиотека находится на пути к добавлению трейта специально для типов, которые могут осмысленно реализовать `poll_next`. Со временем эти асинхронные итераторы (часто называемые потоками, `streams`) могут в итоге получить полноценную языковую поддержку, такую как возможность напрямую итерировать по ним в цикле!

В конечном счёте этот код трудно писать, трудно читать и трудно изменять. Если бы мы захотели добавить, например, обработку ошибок, сложность кода значительно возросла бы. К счастью, есть способ лучше!

## **async/await**

Rust 1.39 дал нам ключевое слово `async` и тесно связанный с ним постфиксный оператор `await`, которые мы использовали в исходном примере в листинге 9-3. Вместе они предоставляют гораздо более удобный механизм для написания асинхронных машин состояний, подобных той, что в листинге 9-5. В частности, они позволяют вам писать код таким образом, что он даже не выглядит как машина состояний!

```
async fn forward<T>(rx: Receiver<T>, tx: Sender<T>) {  
    while let Some(t) = rx.next().await {  
        tx.send(t).await;  
    }  
}
```

*Листинг 9-5: Реализация пересылающей по каналу футуры с помощью `async` и `await`, повторение листинга 9-3*

Если у вас немного опыта с `async` и `await`, разница между листингом 9-4 и листингом 9-5 может дать вам представление о том, почему сообщество Rust так радовалось их появлению. Но поскольку это книга промежуточного уровня, давайте копнём чуть глубже, чтобы понять, как именно этот короткий фрагмент кода может заменить гораздо более длинную ручную реализацию. Для этого нам сначала нужно поговорить о *генераторах* (generators) — механизме, посредством которого реализованы `async` и `await`.

### ***Генераторы (Generators)***

Если описать кратко, генератор (generator) — это фрагмент кода с некоторыми дополнительными битами, генерируемыми компилятором, которые позволяют ему остановиться, или *уступить* (yield), своё выполнение посередине, а затем возобновиться позже с того места, где он в последний раз уступил. Возьмём, к примеру, функцию `forward` из листинга 9-3. Представьте, что она доходит до вызова `send`, но канал в данный момент заполнен. Функция не может продвинуться дальше, но также не может заблокироваться (это же неблокирующий код, в конце концов), так что ей нужно вернуть управление. Теперь предположим, что канал в конце концов освобождается, и мы хотим продолжить отправку. Если мы снова вызовем `forward` с вершины, она снова вызовет `next`, и элемент, который мы ранее пытались отправить, будет потерян, а это никуда не годится. Вместо этого мы превращаем `forward` в генератор.

Всякий раз, когда генератор `forward` больше не может продвигаться вперёд, ему нужно где-то сохранить своё текущее состояние, чтобы, когда его выполнение в конце концов возобновится, оно возобновилось в правильном месте с правильным состоянием. Он сохраняет состояние через ассоциированную структуру данных, которая генерируется компилятором и которая содержит всё состояние генератора в данный момент времени. Метод на этой структуре данных (также сгенерированный компилятором) затем

позволяет генератору возобновиться из текущего состояния, хранящегося в `&mut self`, и снова обновляет состояние, когда генератор снова не может продвинуться вперёд.

Эта операция «верни управление, но позволь мне возобновиться позже» называется *уступанием* (yielding), что фактически означает, что генератор возвращает управление, сохраняя при этом некоторое дополнительное состояние сбоку. Когда мы позже хотим возобновить вызов `forward`, мы вызываем известную точку входа в генератор (*метод возобновления*, `resume method`, который для асинхронных генераторов является `poll`), и генератор изучает ранее сохранённое состояние в `self`, чтобы решить, что делать дальше. Это в точности то же самое, что мы делали вручную в листинге 9-4! Иными словами, код в листинге 9-5 примерно разворачивается (desugars) в гипотетический код, показанный в листинге 9-6.

```
generator fn forward<T>(rx: Receiver<T>, tx: Sender<T>) {
    loop {
        let mut f = rx.next();
        let r = if let Poll::Ready(r) = f.poll() { r } else { yield };
        if let Some(t) = r {
            let mut f = tx.send(t);
            let _ = if let Poll::Ready(r) = f.poll() { r } else { yield };
        } else { break Poll::Ready(()); }
    }
}
```

#### *Листинг 9-6: Разворачивание `async/await` в генератор*

На момент написания генераторы на самом деле непригодны для использования в Rust — они используются только внутренне компилятором для реализации `async/await`, — но в будущем это может измениться. Генераторы пригождаются в ряде случаев, например для реализации итераторов без необходимости таскать с собой структуру или для реализации `impl Iterator`, который сам разбирается, как уступать элементы по одному.

Если вы внимательно посмотрите на листинги 9-5 и 9-6, они могут показаться слегка волшебными, как только вы осознаете, что каждый `await` или `yield` на самом деле является возвратом из функции. В конце концов, в функции есть несколько локальных переменных, и неясно, как они восстанавливаются, когда мы позже возобновляемся. Вот тут-то и вступает в игру сгенерированная компилятором часть генераторов. Компилятор прозрачно

вставляет код, который сохраняет эти переменные в ассоциированную структуру данных генератора и читает их оттуда, а не из стека, во время выполнения. Так что если вы объявляете, записываете или читаете из некоторой локальной переменной, вы на самом деле оперируете чем-то вроде `self.a`. Проблема решена! Всё это довольно изумительно.

Одно тонкое, но важное различие между ручной реализацией `forward` и версией с `async/await` состоит в том, что последняя может удерживать ссылки через точки уступания. Это позволяет таким функциям, как `Receiver::next` и `Sender::send` в листинге 9-5, принимать `&mut self`, а не `self`, который они принимали в листинге 9-4. Если бы мы попытались использовать получатель `&mut self` для этих методов в ручной реализации машины состояний, проверщик заимствований не имел бы способа гарантировать, что на `Receiver`, хранящийся внутри `Forward`, нельзя сослаться между моментом вызова `Receiver::next` и моментом разрешения возвращённой им футуры, и поэтому он отверг бы код. Только переместив `Receiver` в футуру, мы можем убедить компилятор в том, что `Receiver` иным образом недоступен. Между тем при использовании `async/await` проверщик заимствований может проинспектировать код до того, как компилятор превратит его в машину состояний, и проверить, что к `rx` действительно нет повторного доступа до тех пор, пока футура не будет уничтожена, когда возвращается `await` на ней.

**РАЗМЕР ГЕНЕРАТОРОВ (THE SIZE OF GENERATORS)** Структура данных, используемая для хранения состояния генератора, должна быть способна вмещать совокупное состояние в любой одной точке уступания. Если ваша `async fn` содержит, скажем, `[u8; 8192]`, эти 8 КиБ должны храниться в самом генераторе. Даже если ваша `async fn` содержит только меньшие локальные переменные, она всё равно должна содержать любую футуру, которую она ожидает, поскольку ей нужно иметь возможность опросить такую футуру позже, когда вызывается `poll`.

Эта вложенность означает, что генераторы, а значит и футуры, основанные на асинхронных функциях и блоках, могут становиться довольно большими без какого-либо видимого индикатора этого возросшего размера. Это, в свою очередь, может повлиять на производительность вашей программы во время выполнения, поскольку эти гигантские генераторы, возможно, придётся копировать

между вызовами функций и в структуры данных и из них, что в сумме даёт немалый объём копирования памяти. На самом деле, обычно можно определить, когда размер ваших основанных на генераторах футур влияет на производительность, по чрезмерному количеству времени, проводимому в функции `testrun` в профилях производительности вашего приложения!

Однако находить эти большие футуры не всегда легко, и часто это требует ручного выявления длинных или сложных цепочек асинхронных функций. `Clippy`, возможно, сможет помочь с этим в будущем, но на момент написания вы предоставлены сами себе. Когда вы всё же обнаруживаете особенно большую футуру, у вас есть два варианта: вы можете попытаться сократить объём локального состояния, нужного асинхронным функциям, или вы можете переместить футуру в кучу (с помощью `Box::pin`), чтобы перемещение футуры требовало лишь перемещения указателя на неё. Последнее на сегодняшний день самый простой путь, но оно также вносит дополнительное выделение памяти и косвенность указателя. Ваша лучшая ставка — обычно поместить проблемную футуру в кучу, измерить производительность, а затем использовать ваши бенчмарки производительности, чтобы они направляли вас дальше.

## Pin и Unpin

Мы ещё не совсем закончили. Хотя генераторы изящны, из техники, как я её описал до сих пор, возникает одна сложность. В частности, неясно, что происходит, если код в генераторе (или, что эквивалентно, в асинхронном блоке) берёт ссылку на локальную переменную. В коде из листинга 9-5 футура, которую возвращает `rx.next()`, обязательно должна удерживать ссылку на `rx`, если следующее сообщение недоступно сразу, чтобы она знала, откуда повторить попытку, когда генератор в следующий раз возобновится. Когда генератор уступает, футура и ссылка, которую содержит футура, прячутся внутрь генератора. Но что происходит, если генератор перемещается? В частности, посмотрите на код в листинге 9-7, который вызывает `forward`.

```
async fn try_forward<T>(rx: Receiver<T>, tx: Sender<T>) -> Option<impl Future> {
    let mut f = forward(rx, tx);
    if f.poll().is_pending() { Some(f) } else { None }
}
```

### Листинг 9-7: Перемещение футуры после её опроса

Функция `try_forward` опрашивает `forward` только один раз, чтобы переслать столько сообщений, сколько возможно, без блокировки. Если получатель всё ещё может произвести больше сообщений (то есть если он вернул `Poll::Pending`, а не `Poll::Ready(None)`), то эти сообщения откладываются для пересылки в какой-то более поздний момент времени путём возврата пересылающей футуры вызывающему, который может выбрать опросить её снова, когда сочтёт нужным.

Давайте разберём, что здесь происходит, с учётом того, что мы знаем об `async` и `await`. Когда мы опрашиваем генератор `forward`, он проходит цикл `while` некоторое неизвестное число раз и в конце концов возвращает либо `Poll::Ready()`, если получатель закончился, либо `Poll::Pending` в противном случае. Если он возвращает `Poll::Pending`, генератор содержит футуру, возвращённую либо `rx.next()`, либо `tx.send(t)`. Обе эти футуры содержат ссылку на один из аргументов, изначально предоставленных `forward` (`rx` и `tx` соответственно), которые тоже должны храниться в генераторе. Но когда `try_forward` возвращает целый генератор, поля генератора тоже перемещаются. Таким образом, `rx` и `tx` больше не находятся в тех же местах в памяти, и ссылки, хранящиеся в припрятанной футуре, больше не указывают на правильные данные!

То, с чем мы здесь столкнулись, — это случай *самоссылочной* (self-referential) структуры данных: такой, которая хранит как данные, так и ссылки на эти данные. С генераторами эти самоссылочные структуры очень легко сконструировать, и невозможность их поддерживать была бы существенным ударом по эргономике, поскольку это означало бы, что вы не сможете удерживать ссылки через любую точку уступания. (Изобретательное) решение для поддержки самоссылочных структур данных в Rust приходит в форме типа `Pin` и трейта `Unpin`. Очень кратко: `Pin` — это тип-обёртка, который предотвращает (безопасное) перемещение обёрнутого типа, а `Unpin` — это маркерный трейт, который говорит, что реализующий его тип *может* быть безопасно извлечён из `Pin`.

### ***Pin***

Здесь много нюансов, которые нужно охватить, так что давайте начнём с конкретного использования обёртки `Pin`. Листинг 9-2 дал вам упрощённую



версию трейта `Future`, но теперь мы готовы снять одну часть этого упрощения. Листинг 9-8 показывает трейт `Future` несколько ближе к его финальной форме.

```
trait Future {  
    type Output;  
    fn poll(self: Pin<&mut Self>) -> Poll<Self::Output>;  
}
```

*Листинг 9-8: Менее упрощённый взгляд на трейт `Future` с `Pin`*

В частности, это определение требует, чтобы вы вызывали `poll` на `Pin<&mut Self>`. Как только у вас есть значение за `Pin`, это представляет собой контракт, что это значение больше никогда не будет перемещено. Это означает, что вы можете конструировать самоссылки сколько душе угодно, в точности как вам это нужно для генераторов.

**ПРИМЕЧАНИЕ** Хотя `Future` использует `Pin`, `Pin` не привязан к трейту `Future` — вы можете использовать `Pin` для любой самоссылочной структуры данных.

Но как же получить `Pin`, чтобы вызвать `poll`? И как `Pin` может гарантировать, что содержащееся значение не переместится? Чтобы увидеть, как работает эта магия, давайте посмотрим на определение `std::pin::Pin` и некоторые из его ключевых методов, показанные в листинге 9-9.

```
struct Pin<P> { pointer: P }  
impl<P> Pin<P> where P: Deref {  
    pub unsafe fn new_unchecked(pointer: P) -> Self;  
}  
impl<'a, T> Pin<&'a mut T> {  
    pub unsafe fn get_unchecked_mut(self) -> &'a mut T;  
}  
impl<P> Deref for Pin<P> where P: Deref {  
    type Target = P::Target;  
    fn deref(&self) -> &Self::Target;  
}
```

*Листинг 9-9: `std::pin::Pin` и его ключевые методы*

Здесь много что нужно распаковать, и нам придётся пройти по определению из листинга 9-9 несколько раз, прежде чем все детали обретут смысл, так что, пожалуйста, потерпите.

Во-первых, вы заметите, что `Pin` хранит *тип-указатель* (pointer type). То есть



вместо того чтобы хранить какой-то `T` напрямую, он хранит тип `P`, который разыменовывается через `Deref` в `T`. Это означает, что вместо `Pin<MyType>` у вас будет `Pin<Box<MyType>>`, или `Pin<Rc<MyType>>`, или `Pin<&mut MyType>`. Причина такого дизайна проста — основная цель `Pin` в том, чтобы гарантировать, что как только вы помещаете `T` за `Pin`, этот `T` не переместится, поскольку это могло бы инвалидировать самоссылки, хранящиеся в `T`. Если бы `Pin` просто держал `T` напрямую, то простое перемещение `Pin` было бы достаточным, чтобы инвалидировать этот инвариант! В остальной части этого раздела я буду называть `P` *указательным* (pointer) типом, а `T` — *целевым* (target) типом.

Далее обратите внимание, что конструктор `Pin`, `new_unchecked`, небезопасен (unsafe). Это потому, что компилятор не имеет способа фактически проверить, что тип-указатель действительно обещает, что тип, на который он указывает (целевой), больше не переместится. Рассмотрим, например, переменную `foo` на стеке. Если бы конструктор `Pin` был безопасным, мы могли бы сделать `Pin::new(&mut foo)`, вызвать метод, который требует `Pin<&mut Self>` (и, следовательно, предполагает, что `Self` не переместится), а затем уничтожить `Pin`. В этот момент мы могли бы изменять `foo` сколько душе угодно — включая её перемещение! Затем мы могли бы снова закрепить её и вызвать тот же метод, который ни о чём не догадался бы о том, что любые самоссылочные указатели, которые он, возможно, сконструировал в первый раз, теперь стали бы недействительными.

**БЕЗОПАСНОСТЬ КОНСТРУКТОРА PIN (PIN CONSTRUCTOR SAFETY)** Другая причина, по которой конструктор для `Pin` небезопасен, состоит в том, что его безопасность зависит от реализаций трейтов, которые сами по себе безопасны. Например, способ, которым `Pin<P>` реализует `get_unchecked_mut`, — это использовать реализацию `DerefMut::deref_mut` для `P`. Хотя вызов `get_unchecked_mut` небезопасен, реализация `impl DerefMut for P` — нет. Тем не менее она получает `&mut self` и, следовательно, может свободно (и без небезопасного кода) перемещать `T`. То же самое относится к `Drop`. Поэтому требование безопасности для `Pin::new_unchecked` состоит не только в том, что тип-указатель не позволит снова переместить целевой тип (как в примере с `Pin<&mut T>`), но также и в том, что его реализации `Deref`, `DerefMut` и `Drop` не перемещают значение, на которое указывает получаемый ими `&mut self`.

Затем мы доходим до метода `get_unchecked_mut`, который даёт вам изменяемую ссылку на `T` за типом-указателем `Pin`. Этот метод тоже небезопасен, потому что как только мы выдаём `&mut T`, вызывающий должен пообещать, что не будет использовать этот `&mut T` для перемещения `T` или иной инвалидации его памяти, чтобы не сделать недействительными какие-либо самоссылки. Если бы этот метод не был небезопасным, вызывающий мог бы вызвать метод, принимающий `Pin<&mut Self>`, а затем вызвать безопасный вариант `get_unchecked_mut` на двух `Pin<&mut _>`, а затем использовать `mem::swap`, чтобы поменять местами значения за `Pin`. Если бы мы затем снова вызвали метод, принимающий `Pin<&mut Self>`, на любом из `Pin`, его предположение о том, что `Self` не переместился, было бы нарушено, и любые внутренние ссылки, которые он сохранил, стали бы недействительными!

Возможно, удивительно, но `Pin<P>` всегда реализует `Deref<Target = T>`, и это совершенно безопасно. Причина в том, что `&T` не позволяет вам перемещать `T` без написания другого небезопасного кода (например, `UnsafeCell`, как мы обсудим в главе 10). Это хороший пример того, почему область действия небезопасного блока выходит за пределы кода, который он содержит. Если бы вы написали какой-то код в одной части приложения, который (небезопасно) заменил `T` за `&` с помощью `UnsafeCell`, то *могло бы* оказаться, что `T` изначально пришёл из `Pin<&mut T>`, и теперь вы нарушили инвариант о том, что `T` за `Pin` никогда не должен перемещаться, даже несмотря на то, что место, где вы небезопасно заменили `&T`, даже не упоминало `Pin`!

**ПРИМЕЧАНИЕ** Если вы листали документацию `Pin`, читая эту главу, вы, возможно, заметили `Pin::set`, который принимает `&mut self` и `<P as Deref>::Target` и безопасно изменяет значение за `Pin`. Это возможно, потому что `set` не возвращает значение, которое ранее было закреплено, — он просто уничтожает его на месте и сохраняет новое значение вместо него. Поэтому он не нарушает инварианты закрепления (pinning): старое значение никогда не было доступно вне `Pin` после того, как оно было туда помещено.

### ***Unpin: ключ к безопасному закреплению (Unpin: The Key to Safe Pinning)***

На этом этапе вы могли бы спросить: раз получение изменяемой ссылки всё равно небезопасно, почему бы не сделать так, чтобы `Pin` держал `T` напрямую?

То есть вместо того чтобы требовать косвенность через тип-указатель, можно было бы вместо этого сделать контракт для `get_unchecked_mut` таким, что его безопасно вызывать, только если вы не перемещали `Pin`. Ответ на этот вопрос кроется в одном изящном безопасном использовании `Pin`, которое позволяет дизайн указателя. Вспомните, что вся причина, по которой нам в первую очередь нужен `Pin`, в том, чтобы мы могли иметь целевые типы, которые могут содержать ссылки на самих себя (как генератор), и давать их методам гарантию того, что целевой тип не переместился и, следовательно, внутренние самоссылки остаются действительными. `Pin` позволяет нам использовать систему типов для обеспечения этой гарантии, что прекрасно. Но, к сожалению, с дизайном, как он есть до сих пор, `Pin` очень неудобен в работе. Это потому, что он всегда требует небезопасного кода, даже если вы работаете с целевым типом, который не содержит никаких самоссылок и, следовательно, не заботится о том, был он перемещён или нет.

Вот тут-то и вступает в игру маркерный трейт `Unpin`. Реализация `Unpin` для типа просто утверждает, что этот тип безопасно извлекать из `Pin`, когда он используется как целевой тип. То есть тип обещает, что он никогда не будет использовать какие-либо гарантии `Pin` о том, что референс больше не перемещается, когда он используется как целевой тип, и, следовательно, эти гарантии могут быть нарушены. `Unpin` — это авто-трейт (auto-trait), как `Send` и `Sync`, и поэтому автоматически реализуется компилятором для любого типа, который содержит только члены, реализующие `Unpin`. Только типы, которые явно отказываются от `Unpin` (как генераторы), и типы, содержащие такие типы, являются `!Unpin`.

Для целевых типов, которые являются `Unpin`, мы можем предоставить гораздо более простой безопасный интерфейс к `Pin`, как показано в листинге 9-10.

```
impl<P> Pin<P> where P: Deref, P::Target: Unpin {
    pub fn new(pointer: P) -> Self;
}
impl<P> DerefMut for Pin<P> where P: DerefMut, P::Target: Unpin {
    fn deref_mut(&mut self) -> &mut Self::Target;
}
```

*Листинг 9-10: Безопасный API к `Pin` для целевых типов `Unpin`*

Чтобы осмыслить безопасный API в листинге 9-10, подумайте о требованиях безопасности небезопасных методов из листинга 9-9. Функция

`Pin::new_unchecked` небезопасна, потому что вызывающий должен обещать, что референт нельзя перемещать вне `Pin` и что реализации `Deref`, `DerefMut` и `Drop` для типа-указателя не перемещают референт через ссылку, которую они получают. Эти требования существуют, чтобы гарантировать, что как только мы выдаём `Pin` на `T`, мы больше никогда не перемещаем этот `T`. Но если `T` является `Unpin`, он заявил, что ему всё равно, перемещают ли его, даже если он ранее был закреплён, так что всё в порядке, если вызывающий не удовлетворяет ни одному из этих требований!

Аналогично, `get_unchecked_mut` небезопасен, потому что вызывающий должен гарантировать, что не перемещает `T` из `&mut T` — но при `T: Unpin` `T` заявил, что для него нормально быть перемещённым даже после закрепления, так что это требование безопасности больше не важно. Это означает, что для `Pin<P>`, где `P::Target: Unpin`, мы можем просто предоставить безопасные варианты обоих этих методов (`DerefMut` будучи безопасной версией `get_unchecked_mut`). На самом деле мы можем даже предоставить `Pin::into_inner`, который просто возвращает владеемое `P`, если целевой тип является `Unpin`, поскольку тогда `Pin` по сути нерелевантен!

### ***Способы получения Pin (Ways of Obtaining a Pin)***

С нашим новым пониманием `Pin` и `Unpin` мы теперь можем продвинуться к использованию нового определения `Future` из листинга 9-8, которое требует `Pin<&mut Self>`. Первый шаг — сконструировать требуемый тип. Если тип футуры является `Unpin`, этот шаг прост — мы просто используем `Pin::new(&mut future)`. Если он не `Unpin`, мы можем закрепить футуру одним из двух основных способов: закреплением в куче или закреплением на стеке.

Давайте начнём с закрепления в куче. Основной контракт `Pin` состоит в том, что как только что-то закреплено, оно не может переместиться. API закрепления берёт на себя соблюдение этого контракта для всех методов и трейтов на `Pin`, так что главная роль любой функции, которая конструирует `Pin`, состоит в том, чтобы гарантировать, что если *сам* `Pin` перемещается, референт не перемещается тоже. Самый простой способ гарантировать это — поместить референт в кучу, а затем поместить в `Pin` лишь указатель на референт. После этого вы можете перемещать `Pin` сколько душе угодно, а цель останется на месте. Это и есть обоснование (безопасного) метода `Box::pin`, который принимает `T` и возвращает `Pin<Box<T>>`. В нём нет никакой магии; он

просто утверждает, что `Box` следует контрактам конструктора `Pin`, `Deref` и `Drop`.

**UNPIN BOX (UNPIN BOX)** Раз уж мы коснулись темы `Box`, взгляните на реализацию `Unpin` для `Box`. Тип `Box` безусловно реализует `Unpin` для любого `T`, даже если этот `T` не является `Unpin`. Это может показаться вам странным, учитывая ранее сделанное утверждение о том, что `Unpin` — авто-трейт, который в общем случае реализуется для типа только в том случае, если все члены типа тоже являются `Unpin`. `Box` является исключением из этого по той же причине, по которой он может предоставить безопасный конструктор `Pin`: если вы перемещаете `Box<T>`, вы не перемещаете `T`. Иными словами, безусловная реализация утверждает, что вы можете извлечь `Box<T>` из `Pin`, даже если `T` нельзя извлечь из `Pin`. Однако заметьте, что это *не* позволяет вам извлечь `T`, который является `!Unpin`, из `Pin<Box<T>>`.

Другой вариант, закрепление на стеке, немного сложнее и на момент написания требует крупицы небезопасного кода. Мы должны гарантировать, что к закреплённому значению нельзя получить доступ после того, как `Pin` с `&mut` на него был уничтожен. Мы достигаем этого затенением (shadowing) значения, как показано в макросе в листинге 9-11, или используя один из крейтов, которые предоставляют именно этот макрос. Однажды он, возможно, даже попадёт в стандартную библиотеку!

```
macro_rules! pin_mut {
  ($var:ident) => {
    let mut $var = $var;
    let mut $var = unsafe { Pin::new_unchecked(&mut $var) };
  }
}
```

#### *Листинг 9-11: Макрос для закрепления на стеке*

Принимая имя переменной для закрепления на стеке, макрос гарантирует, что у вызывающего уже есть значение, которое он хочет закрепить, где-то на стеке. Затенение `$var` гарантирует, что вызывающий не сможет уничтожить `Pin` и продолжить использовать незакреплённое значение (что нарушило бы контракт `Pin` для любого целевого типа, который является `!Unpin`). Перемещая значение, хранящееся в `$var`, макрос также гарантирует, что вызывающий не сможет уничтожить привязку `$var` без одновременного уничтожения исходной переменной. В частности, без этой строки вызывающий мог бы написать

(обратите внимание на дополнительную область видимости):

```
let foo = /* */; { pin_mut!(foo); foo.poll() }; foo.mut_self_method();
```

Здесь мы передаём закреплённый экземпляр `foo` в `poll`, но затем позже используем `&mut foo` без повторного закрепления, что нарушает контракт `Pin`. С дополнительным переприсваиванием, с другой стороны, этот код также переместил бы `foo` в новую область видимости, сделав её непригодной для использования после окончания области видимости.

Закрепление на стеке поэтому требует небезопасного кода, в отличие от `Box::pin`, но избегает дополнительного выделения памяти, которое вносит `Box`, а также работает в окружениях `no_std`.

## Назад в будущее (Back to the Future)

Теперь у нас есть наша закреплённая футура, и мы знаем, что это означает. Но вы, возможно, заметили, что ни один из этих важных аспектов закрепления не появляется в большей части асинхронного кода, который вы пишете с `async` и `await`. И это потому, что компилятор скрывает их от вас.

Вспомните момент, когда мы обсуждали листинг 9-5 и я сказал вам, что `<expr>.await` разворачивается в нечто вроде:

```
loop { if let Poll::Ready(r) = expr.poll() { break r } else { yield } }
```

Это было совсем уж лёгкое упрощение, поскольку, как мы видели, вы можете вызвать `Future::poll`, только если у вас есть `Pin<&mut Self>` для футуры. Разворачивание на самом деле немного более изощрённое, как показано в листинге 9-12.

```
match expr { ❶
  mut pinned => loop {
    match unsafe { Pin::new_unchecked(&mut pinned) }.poll() { ❷
      Poll::Ready(r) => break r,
      Poll::Pending => yield,
    }
  }
}
```

*Листинг 9-12: Более точное разворачивание `<expr>.await`*

match ❶ — это аккуратное сокращение, которое не только гарантирует, что разворачивание остаётся валидным выражением, но также перемещает результат выражения в переменную, которую мы затем можем закрепить на стеке. Помимо этого, главное новое добавление — это вызов `Pin::new_unchecked` ❷. Этот вызов безопасен, потому что для того чтобы содержащий его асинхронный блок мог быть опрошен, он уже должен быть закреплён в силу сигнатуры `Future::poll`. А асинхронный блок был опрошен для того, чтобы мы достигли вызова `Pin::new_unchecked`, так что состояние генератора закреплено. Поскольку `pinned` хранится в генераторе, соответствующем асинхронному блоку (это должно быть так, чтобы `yield` корректно возобновлялся), мы знаем, что `pinned` не переместится. Более того, к `pinned` нельзя получить доступ нигде, кроме как через `Pin` в цикле, в котором мы находимся, так что ни один код не способен переместить значение из `pinned`. Таким образом, мы удовлетворяем всем требованиям безопасности `Pin::new_unchecked`, и код безопасен.

## Засыпание (Going to Sleep)

Мы довольно глубоко погрузились в дебри с `Pin`, но теперь, когда мы выбрали с другой стороны, есть ещё одна проблема, связанная с футурами, которая, возможно, не давала вам покоя. Если вызов `Future::poll` возвращает `Poll::Pending`, вам нужно что-то, что снова вызовет `poll` в более поздний момент, чтобы проверить, можете ли вы уже продвинуться вперёд. Это что-то обычно называется *исполнителем* (executor). Ваш исполнитель мог бы быть простым циклом, который опрашивает все футуры, которых вы ждёте, пока все они не вернут `Poll::Ready`, но это сожгло бы массу процессорных циклов, которые вы, вероятно, могли бы использовать с большей пользой, например для работы вашего веб-браузера. Вместо этого мы хотим, чтобы исполнитель делал всю полезную работу, какую может, а затем засыпал. Он должен оставаться в спящем состоянии, пока одна из футур не сможет продвинуться вперёд, и только тогда проснуться, чтобы сделать ещё один проход, прежде чем снова заснуть.

## Пробуждение (Waking Up)

Условие, которое определяет, когда нужно вернуться к проверке данной футуры, сильно варьируется. Это может быть «когда на этот порт придёт сетевой пакет», «когда переместится курсор мыши», «когда кто-то отправит в



этот канал», «когда процессор получит определённое прерывание» или даже «по истечении вот такого-то промежутка времени». Вдобавок ко всему, разработчики могут писать собственные футуры, которые оборачивают несколько других футур, и, следовательно, у них может быть несколько условий пробуждения. Некоторые футуры могут даже вводить свои полностью кастомные события пробуждения.

Чтобы вместить все эти случаи использования, Rust вводит понятие *пробуждателя* (Waker): способ разбудить исполнитель, чтобы сигнализировать, что прогресс может быть достигнут. Waker — это то, что заставляет работать всю машинерию вокруг футур. Исполнитель конструирует Waker, который интегрируется с механизмом, который исполнитель использует для засыпания, и передаёт Waker в любую Future, которую он опрашивает. Как? С помощью дополнительного параметра `Future::poll`, который я скрывал от вас до сих пор. Прощу прощения за это. Листинг 9-13 даёт финальное и истинное определение Future — больше никакой лжи!

```
trait Future {  
    type Output;  
    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;  
}
```

#### *Листинг 9-13: Настоящий трейт Future с Context*

`&mut Context` содержит Waker. Аргумент — это Context, а не Waker напрямую, чтобы мы могли дополнить асинхронную экосистему дополнительным контекстом для футур, если это будет сочтено необходимым.

Основной метод на Waker — это `wake` (и вариант по ссылке `wake_by_ref`), который следует вызывать, когда футура снова может продвинуться вперёд. Метод `wake` не принимает аргументов, и его эффекты полностью определяются исполнителем, который сконструировал Waker. Видите ли, Waker втайне обобщён по исполнителю. Точнее, то, что сконструировало Waker, диктует, что происходит при вызове `Waker::wake`, при клонировании Waker и при уничтожении Waker. Всё это происходит через вручную реализованную виртуальную таблицу (vtable), которая функционирует подобно динамической диспетчеризации, которую мы обсуждали ещё в главе 3.

Это несколько запутанный процесс — сконструировать Waker, и механика его не так уж важна для его использования, но вы можете увидеть строительные



блоки в типе `RawWakerVTable` в стандартной библиотеке. У него есть конструктор, который принимает указатели на функции для `wake` и `wake_by_ref`, а также для `Clone` и `Drop`. `RawWakerVTable`, которая обычно разделяется между всеми пробуждателями исполнителя, упаковывается вместе с сырым указателем, предназначенным для хранения данных, специфичных для каждого экземпляра `Waker` (например, для какой футуры он предназначен), и превращается в `RawWaker`. Это, в свою очередь, передаётся в `Waker::from_raw`, чтобы получить безопасный `Waker`, который может быть передан в `Future::poll`.

## Выполнение контракта опроса (Fulfilling the Poll Contract)

До сих пор мы обходили стороной вопрос о том, что футура на самом деле делает с `Waker`. Идея довольно проста: если `Future::poll` возвращает `Poll::Pending`, то ответственность футуры состоит в том, чтобы гарантировать, что *что-то* вызовет `wake` на предоставленном `Waker`, когда футура в следующий раз сможет продвинуться вперёд. Большинство футур поддерживают это свойство, возвращая `Poll::Pending` только тогда, когда какая-то другая футура тоже вернула `Poll::Pending`; таким образом она тривиально выполняет контракт `poll`, поскольку внутренняя футура должна следовать тому же контракту. Но не может же быть черепах до самого низа. В какой-то момент вы доходите до футуры, которая не опрашивает другие футуры, а вместо этого делает что-то вроде записи в сетевой сокет или попытки приёма из канала. Такие футуры обычно называют *листовыми футурами* (leaf futures), поскольку у них нет детей. У листовой футуры нет внутренней футуры, а вместо этого она напрямую представляет некоторый ресурс, который может быть пока не готов вернуть результат.

**ПРИМЕЧАНИЕ** Контракт `poll` — это причина, по которой рекурсивный вызов `poll` ❹ в листинге 9-4 необходим для корректности.

Листовые футуры обычно бывают одной из двух форм: те, что ждут событий, которые возникают внутри того же процесса (как получатель канала), и те, что ждут событий, внешних по отношению к процессу (как чтение из TCP-пакета). Те, что ждут внутренних событий, обычно следуют одному и тому же шаблону: сохранить `Waker` там, где код, который вас разбудит, сможет его найти, и сделать так, чтобы этот код вызвал `wake` на `Waker`, когда сгенерирует соответствующее событие. Например, рассмотрим листовую футуру, которой нужно ждать сообщения в канале в памяти. Она сохраняет свой `Waker` внутри

той части канала, которая разделяется между отправителем и получателем, а затем возвращает `Poll::Pending`. Когда отправитель позже приходит и впрыскивает сообщение в канал, он замечает `Waker`, оставленный там ожидающим получателем, и вызывает `wake` на `Waker`, прежде чем вернуться из `send`. Теперь получатель пробуждён, и контракт `poll` соблюлён.

Листовые футуры, которые имеют дело с внешними событиями, более сложны, поскольку код, который генерирует событие, которого они ждут, ничего не знает о футурах или пробуждателях. Чаще всего генерирующий код — это ядро операционной системы, которое знает, когда диск готов или истёк таймер, но это может быть и Си-библиотека, которая вызывает обратный вызов (callback) в Rust, когда операция завершается, или какая-то иная подобная внешняя сущность. Листовая футура, обращающаяся внешний ресурс подобного рода, могла бы запустить поток, который выполняет блокирующий системный вызов (или ждёт Си-обратного вызова), а затем использует внутренний механизм пробуждения, но это было бы расточительно; вы запускали бы поток каждый раз, когда операции нужно подождать, и оставались бы с массой одноразовых потоков, сидящих без дела в ожидании событий.

Вместо этого исполнители обычно предоставляют реализации листовых футур, которые за кулисами общаются с исполнителем, чтобы организовать соответствующее взаимодействие с операционной системой. Как именно это оркеструется, зависит от исполнителя и операционной системы, но грубо говоря, исполнитель отслеживает все источники событий, которые он должен прослушивать, когда в следующий раз заснёт. Когда листовая футура осознаёт, что должна подождать внешнего события, она обновляет состояние этого исполнителя (которое она знает, поскольку оно предоставлено крейтом исполнителя), чтобы включить внешний источник событий вместе со своим `Waker`. Когда исполнитель больше не может продвигаться вперёд, он собирает все источники событий, которых ждут различные ожидающие листовые футуры, и делает один большой блокирующий вызов к операционной системе, говоря ей вернуть управление, когда у *любого* из ресурсов, которых ждут листовые футуры, появится новое событие. В Linux это обычно достигается с помощью системного вызова `epoll`; Windows, BSD, macOS и практически любая другая операционная система предоставляют похожие механизмы. Когда этот вызов возвращает управление, исполнитель вызывает `wake` на всех пробуждателях, связанных с источниками событий, по которым операционная

система сообщила о событиях, и таким образом контракт `poll` выполняется.

**ПРИМЕЧАНИЕ** Реактор (reactor) — это часть исполнителя, в которой листовые футуры регистрируют источники событий и на которой исполнитель ждёт, когда у него больше нет полезной работы. Возможно разделить исполнитель и реактор, хотя объединение их вместе часто улучшает производительность, поскольку их легче совместно оптимизировать.

Побочный эффект тесной интеграции между листовыми футурами и исполнителем состоит в том, что листовые футуры из одного крейта-исполнителя часто нельзя использовать с другим исполнителем. Или, по крайней мере, их нельзя использовать, если исполнитель листовой футуры *тоже* не запущен. Когда листовая футура идёт сохранять свой `Waker` и регистрировать источник события, которого она ждёт, исполнителю, под который она была написана, нужно, чтобы это состояние было настроено, и нужно, чтобы он был запущен, чтобы источник события действительно отслеживался и в конце концов вызывался `wake`. Есть способы обойти это, например заставить листовые футуры запускать исполнитель, если он ещё не запущен, но это не всегда целесообразно, поскольку означает, что приложение может незаметно для себя получить несколько одновременно работающих исполнителей, что может снизить производительность и означать, что при отладке вам придётся инспектировать состояние нескольких исполнителей.

Библиотечные крейты, которые хотят поддерживать несколько исполнителей, должны быть обобщёнными по своим листовым ресурсам. Например, вместо использования конкретного типа футуры `TcpStream` или `File` исполнителя библиотека может хранить обобщённый `T: AsyncRead + AsyncWrite`. Однако экосистема ещё не пришла к окончательному соглашению о том, как именно должны выглядеть эти трейты и какие трейты нужны, так что на данный момент довольно трудно сделать код по-настоящему обобщённым по исполнителю. Например, хотя `AsyncRead` и `AsyncWrite` довольно распространены в экосистеме (или их легко адаптировать при необходимости), в настоящее время не существует трейтов для запуска футуры в фоновом режиме (*порождение*, `spawning`, которое мы обсудим позже) или для представления таймера.

## Пробуждение — неточное название (Waking Is a Misnomer)

Возможно, вы уже осознали, что `Waker::wake` не обязательно как будто что-то *пробуждает*. Например, для внешних событий (как описано в предыдущем разделе) исполнитель уже бодрствует, и может показаться глупым, что он затем вызывает `wake` на `Waker`, который всё равно принадлежит этому исполнителю! Реальность в том, что `Waker::wake` — это немного неточное название: на самом деле он сигнализирует, что определённая футура *готова к выполнению* (runnable). То есть он говорит исполнителю, что тот должен убедиться, что опросит эту конкретную футуру, когда дойдёт до неё, а не снова заснёт, поскольку эта футура может продвинуться вперёд. Это может разбудить исполнитель, если он в данный момент спит, чтобы он пошёл опросить эту футуру, но это скорее побочный эффект, чем его основная цель.

Исполнителю важно знать, какие футуры готовы к выполнению, по двум причинам. Во-первых, ему нужно знать, когда он может прекратить опрашивать футуру и заснуть; недостаточно просто опрашивать одну футуру, пока она не вернёт `Poll::Pending`, поскольку опрос более поздней футуры может сделать возможным продвижение более ранней футуры. Рассмотрим случай, когда две футуры перебрасывают сообщения туда-сюда по каналам друг другу. Когда вы опрашиваете одну, другая становится готова, и наоборот. В этом случае исполнитель никогда не должен засыпать, поскольку всегда есть ещё работа.

Во-вторых, знание того, какие футуры готовы к выполнению, позволяет исполнителю избегать ненужного опроса футур. Если исполнитель управляет тысячами ожидающих футур, он не должен опрашивать их все только потому, что некоторое событие сделало одну из них готовой к выполнению. Если бы он это делал, выполнение асинхронного кода стало бы поистине очень медленным.

## Задачи и субисполнители (Tasks and Subexecutors)

Футуры в асинхронной программе образуют дерево: футура может содержать любое число других футур, которые, в свою очередь, могут содержать ещё другие футуры, вплоть до листовых футур, которые взаимодействуют с пробуждателями. Корень каждого дерева — это та футура, которую вы передаёте главной функции «`run`» исполнителя. Эти корневые футуры называются *задачами* (tasks), и они являются единственной точкой контакта

между исполнителем и деревом футур. Исполнитель вызывает `poll` на задаче, и оттуда код каждой содержащейся футуры должен сам разобраться, какую внутреннюю футуру (или футуры) опрашивать в ответ, вплоть до соответствующей листовой.

Исполнители обычно конструируют отдельный `Waker` для каждой задачи, которую они опрашивают, чтобы, когда позже вызывается `wake`, они знали, какая задача только что стала готова к выполнению, и могли пометить её как таковую. Именно для этого предназначен сырой указатель в `RawWaker` — для различения задач при разделении кода между различными методами `Waker`.

Когда исполнитель в конце концов опрашивает задачу, эта задача начинает выполняться с вершины своей реализации `Future::poll` и должна оттуда решить, как добраться до той футуры дальше по дереву, которая теперь может продвинуться вперёд. Поскольку каждая футура знает только о своих собственных полях и ничего о дереве в целом, всё это происходит через вызовы `poll`, каждый из которых проходит одно ребро в дереве.

Выбор того, какую внутреннюю футуру опрашивать, часто очевиден, но не всегда. В случае с `async/await` футура для опроса — это та, которой мы заблокированы в ожидании прогресса. Но в футуре, которая ждёт прогресса первой из нескольких футур (часто называется *select*), или всех из набора футур (часто называется *join*), вариантов много. Футура, которой приходится делать такой выбор, по сути является *субисполнителем* (*subexecutor*). Она могла бы опросить все свои внутренние футуры, но это могло бы быть весьма расточительно. Вместо этого эти субисполнители часто оборачивают `Waker`, который они получают в `Context` метода `poll`, в свой собственный тип `Waker`, прежде чем вызвать `poll` на какой-либо внутренней футуре. В оборачивающем коде они помечают только что опрошенную ими футуру как готовую к выполнению в своём собственном состоянии, прежде чем вызвать `wake` на исходном `Waker`. Таким образом, когда исполнитель в конце концов снова опрашивает футуру-субисполнитель, субисполнитель может свериться со своим собственным внутренним состоянием, чтобы выяснить, какая из его внутренних футур вызвала текущий вызов `poll`, и затем опросить только их.

**БЛОКИРОВКА В АСИНХРОННОМ КОДЕ (BLOCKING IN ASYNC CODE)** Вы должны быть осторожны с вызовом синхронного кода из асинхронного кода, поскольку любое время, которое поток исполнителя тратит на выполнение текущей задачи, — это время,

которое он *не* тратит на выполнение других задач. Если задача занимает текущий поток в течение продолжительного периода времени, не уступая управление обратно исполнителю, что могло бы произойти при выполнении блокирующего системного вызова (как `std::sync::sleep`), при запуске субисполнителя, который не уступает периодически, или при выполнении плотного цикла без `await`, тогда другие задачи, за выполнение которых отвечает текущий поток исполнителя, не получают шанса выполниться в течение этого времени. Обычно это проявляется как длительные задержки между тем, когда определённые задачи могут продвинуться вперёд (например, когда подключается клиент), и тем, когда они фактически получают возможность выполниться.

Некоторые многопоточные исполнители реализуют техники *кражи работы* (work-stealing), где простаивающие потоки исполнителя крадут задачи у занятых потоков исполнителя, но это скорее смягчение, чем решение. В конечном счёте вы можете оказаться в ситуации, когда все потоки исполнителя заблокированы и, следовательно, ни одна задача не выполняется, пока одна из блокирующих операций не завершится.

В общем случае вы должны быть очень осторожны с выполнением вычислительно-интенсивных операций или вызовом функций, которые могут заблокироваться, в асинхронном контексте. Такие операции следует либо преобразовать в асинхронные операции там, где это возможно, либо выполнять на выделенных потоках, которые затем общаются с помощью примитива, поддерживающего асинхронность, как канал. Некоторые исполнители также предоставляют механизмы для указания того, что определённый сегмент асинхронного кода может заблокироваться, или для добровольного уступания управления в циклах, которые иначе не уступали бы, что может составить часть решения. Хорошее эмпирическое правило состоит в том, что ни одна фyuтyра не должна выполняться дольше 1 мс, не вернув `Poll::Pending`.

## Связываем всё воедино с помощью `spawn` (Tying It All Together with `spawn`)

Работая с асинхронными исполнителями, вы можете натолкнуться на операцию, которая порождает (spawns) фyuтyру. Теперь мы в состоянии

разобраться, что это означает! Давайте сделаем это на примере. Сначала рассмотрим простую реализацию сервера в листинге 9-14.

```
async fn handle_client(socket: TcpStream) -> Result<()> {
    // Взаимодействуем с клиентом через заданный сокет.
}

async fn server(socket: TcpListener) -> Result<()> {
    while let Some(stream) = socket.accept().await? {
        handle_client(stream).await?;
    }
}
```

#### *Листинг 9-14: Последовательная обработка соединений*

Функция верхнего уровня `server` по сути является одной большой футурой, которая прослушивает новые соединения и делает что-то, когда приходит новое соединение. Вы передаёте эту футуру исполнителю и говорите «выполни это», и поскольку вы, вероятно, не хотите, чтобы ваша программа сразу же завершилась, исполнитель, скорее всего, заблокируется на этой футуре. То есть вызов исполнителя для запуска футуры `server` не вернёт управление, пока футура `server` не разрешится, что может не произойти никогда (другой клиент всегда может прийти позже).

Теперь, каждый раз, когда приходит новое клиентское соединение, код в листинге 9-14 создаёт новую футуру (вызывая `handle_client`) для обработки этого соединения. Поскольку обработка сама по себе является футурой, мы делаем `await` на ней, а затем переходим к следующему клиентскому соединению.

Недостаток этого подхода в том, что мы всегда обрабатываем только одно соединение за раз — нет конкурентности. Как только сервер принимает соединение, вызывается функция `handle_client`, и поскольку мы делаем `await` на ней, мы не вернёмся к началу цикла, пока возвращённая `handle_client` футура не разрешится (предположительно, когда этот клиент уйдёт).

Мы могли бы улучшить это, храня набор всех клиентских футур и сделав так, чтобы цикл, в котором сервер принимает новые соединения, также проверял все клиентские футуры, чтобы посмотреть, может ли какая-то из них продвинуться. Листинг 9-15 показывает, как это могло бы выглядеть.



```

async fn server(socket: TcpListener) -> Result<()> {
    let mut clients = Vec::new();
    loop {
        poll_client_futures(&mut clients)?;
        if let Some(stream) = socket.try_accept()? {
            clients.push(handle_client(stream));
        }
    }
}

```

### *Листинг 9-15: Обработка соединений с помощью ручного исполнителя*

Это по крайней мере обрабатывает много соединений конкурентно, но это довольно неаккуратно. Это также не очень эффективно, поскольку код теперь крутится в активном цикле, переключаясь между обработкой соединений, которые у нас уже есть, и приёмом новых. И ему приходится проверять каждое соединение каждый раз, поскольку он не знает, какие из них могут продвинуться (если вообще могут). Он также не может делать `await` ни в какой точке, поскольку это помешало бы другим футурам продвигаться. Вы могли бы реализовать собственные пробуждатели, чтобы гарантировать, что код опрашивает только те футуры, которые могут продвинуться, но в конечном счёте это путь к разработке собственного мини-исполнителя.

Другой недостаток использования лишь одной задачи для сервера, которая внутренне содержит футуры для всех клиентских соединений, состоит в том, что сервер в итоге оказывается однопоточным. Есть лишь одна задача, и чтобы опросить её, код должен удерживать эксклюзивную ссылку на футуру задачи (`poll` принимает `Pin<&mut Self>`), которую может удерживать только один поток за раз.

Решение состоит в том, чтобы сделать каждую клиентскую футуру своей собственной задачей и предоставить исполнителю мультиплексировать их среди всех задач. Что, как вы догадались, вы делаете путём порождения футуры. Исполнитель продолжит блокироваться на футуре `server`, но если он не может продвинуться по этой футуре, он будет использовать свою машинерию выполнения, чтобы тем временем продвинуть прогресс по другим задачам за кулисами. И что лучше всего, если исполнитель многопоточный, а ваши клиентские футуры являются `Send`, он может выполнять их параллельно, поскольку может удерживать `&mut` к отдельным задачам конкурентно. Листинг 9-16 даёт пример того, как это могло бы выглядеть.



```
async fn server(socket: TcpListener) -> Result<()> {
    while let Some(stream) = socket.accept().await? {
        // Порождаем новую задачу с Future, которая представляет этого клиента.
        // Текущая задача продолжит просто опрашивать новые соединения
        // и будет выполняться конкурентно (и, возможно, параллельно) с
        handle_client.
        spawn(handle_client(stream));
    }
}
```

*Листинг 9-16: Порождение футур для создания большого числа задач, которые могут опрашиваться конкурентно*

Когда вы порождаете футуру и тем самым делаете её задачей, это в некотором роде похоже на порождение потока. Футура продолжает работать в фоновом режиме и мультиплексируется конкурентно с любыми другими задачами, переданными исполнителю. Однако, в отличие от порождённого потока, порождённые задачи всё ещё зависят от того, чтобы их опрашивал исполнитель. Если исполнитель прекращает работу, либо потому что вы его уничтожили, либо потому что ваш код больше не выполняет код исполнителя, эти порождённые задачи перестанут продвигаться вперёд. В примере с сервером представьте, что произойдёт, если главная футура `server` по какой-то причине разрешится. Поскольку исполнитель вернул управление обратно вашему коду, он не может продолжать делать, ну, хоть что-нибудь. Многопоточные исполнители часто порождают фоновые потоки, которые продолжают опрашивать задачи, даже если исполнитель уступает управление обратно коду пользователя, но не все исполнители это делают, так что проверьте свой исполнитель, прежде чем полагаться на такое поведение!

## Итоги (Summary)

В этой главе мы заглянули за кулисы асинхронных конструкций, доступных в Rust. Мы увидели, как компилятор реализует генераторы и самоссылочные типы и почему эта работа была необходима для поддержки того, что мы теперь знаем как `async/await`. Мы также изучили, как футуры выполняются и как пробуждатели позволяют исполнителям мультиплексировать задачи, когда в данный момент только некоторые из них могут продвинуться вперёд. В следующей главе мы возьмёмся за то, что, пожалуй, является самой глубокой и наиболее обсуждаемой областью Rust: небезопасный код. Сделайте глубокий вдох, а затем переверните страницу.